Data Gap Analysis & Infor Integration Guide

Project: Procurement Analytics Agent
Client: Manufacturing Plant 21C
Date: April 2026
Version: 1.0

---

1. CURRENT STATE ANALYSIS

1.1 Available Data Summary

You currently have **427 materials** with the following data points:

Data Element	Coverage	Quality	Usability
Material Code	100%	✅ High	Excellent
Description	100%	✅ High	Excellent
3-Year Consumption (FY wise)	100%	✅ High	Excellent
Safety Stock	100%	✅ High	Excellent
Lead Time	100%	⚠️ Medium	Good (static values)
Total Records	427 materials × 3 years = 1,281 consumption records		

Strengths:

- Clean, structured data
- Complete historical consumption (3 years)
- No missing values in critical fields
- Good for trend analysis and forecasting

Limitations:

- No monthly/weekly granularity
- No current inventory levels
- No supplier information
- No purchase order data

- No cost/price information
- Static lead times (no variance tracking)

---

2. DATA GAP MATRIX

2.1 Gap Analysis by Question Type

User Question Type	Data Needed	Available?	Source (Infor)	Impact if Missing
"Show slow-moving materials in last 6 months"	Monthly inventory movement	❌ No	ICTRANS, ICDISTRIBUTION	Cannot answer - need granular movement data
"Why has inventory value increased this quarter?"	Current stock levels + Material cost	❌ No	ICLOCATION, ICITEM (unit_cost)	Cannot answer - need current stock and pricing
"Which suppliers frequently delay deliveries?"	PO data + GRN data + Supplier master	❌ No	POLINE, PORELEASEDETAIL, APVENDOR	Cannot answer - need supplier performance tracking
"Which materials are currently critical to production?"	Current stock + Production orders + BOM	❌ No	ICLOCATION, SFORDER, ENGBOM	Cannot answer - need real-time stock and production data
"What's the monthly consumption trend for steel materials?"	Monthly consumption + Material category	⚠️ Partial	Need monthly breakdown	Can estimate from yearly data but not accurate
"Show top 10 consumers in 2024-25"	Yearly consumption	✅ Yes	Already available	✅ Can answer
"Which materials have high YoY growth?"	Multi-year consumption	✅ Yes	Already available	✅ Can answer

Key Insight: You can answer ~30% of business questions with current data. To reach 90%+ coverage, you need the Priority 1 data sources below.

---

3. PRIORITY 1 DATA REQUIREMENTS (CRITICAL)

3.1 Current Inventory Snapshot

Why Critical: Without current stock levels, you cannot identify actual shortages, excess inventory, or real stockout risks.

Required Fields

Field Name	Infor Column	Data Type	Example	Business Use
Material_Code	ITEM	VARCHAR(50)	10401	Material identifier
Plant_Code	COMPANY	VARCHAR(10)	21C	Plant/location
Storage_Location	LOCATION	VARCHAR(20)	MAIN, WH01	Warehouse/bin
Unrestricted_Stock	ON_HAND	DECIMAL(15,3)	1250.500	Available for use
Quality_Inspection_Stock	QI_STOCK	DECIMAL(15,3)	50.000	Under inspection
Blocked_Stock	BLOCKED	DECIMAL(15,3)	0.000	Unusable stock
Stock_in_Transit	IN_TRANSIT	DECIMAL(15,3)	200.000	In shipment
Reserved_Stock	COMMITTED	DECIMAL(15,3)	100.000	Allocated to orders
Available_Stock	AVAILABLE	DECIMAL(15,3)	1150.500	Free to use
Stock_Value	STOCK_VALUE	DECIMAL(15,2)	125050.00	Total value
Unit_of_Measure	UM	VARCHAR(10)	EA, KG, L	Measurement unit
Last_Movement_Date	LAST_MOVE_DATE	DATE	2026-04-01	Last transaction
Snapshot_Date	SYSTEM_DATE	DATE	2026-04-02	Data extraction date

Infor Tables

Primary Table: `ICLOCATION` (Item Location/Balance)

```sql
```

```sql
SELECT
    l.ITEM as material_code,
    l.COMPANY as plant_code,
    l.LOCATION as storage_location,
    l.ON_HAND as unrestricted_stock,
    l.ON_ORDER as on_order_qty,
    l.COMMITTED as reserved_stock,
    (l.ON_HAND - l.COMMITTED) as available_stock,
    l.LAST_MOVE_DATE as last_movement_date,
    CURRENT_DATE as snapshot_date
FROM ICLOCATION l
WHERE l.COMPANY = '21C'
  AND l.ON_HAND > 0  -- Only active inventory
ORDER BY l.ITEM
```

Supporting Table: `ICITEM` (Item Master)

```sql
SELECT
    i.ITEM as material_code,
    i.DESCRIPTION as description,
    i.UM as unit_of_measure,
    i.UNIT_COST as unit_cost,
    i.STD_COST as standard_cost,
    i.ITEM_TYPE as material_type,
    i.ABC_CODE as abc_class
FROM ICITEM i
WHERE i.COMPANY = '21C'
```

Data Refresh Strategy

- **Frequency:** Daily (batch) + Real-time API for critical queries
- **Timing:** Run at 6:00 AM (before business hours)
- **Method:** Incremental load (only changed records)
- **Retention:** Keep 90 days of daily snapshots for trend analysis

Sample Data Expected (Top 5 Materials)

material_code	storage_location	unrestricted_stock	available_stock	stock_value	last_movement
10401	MAIN	1250.50	1150.50	125050.00	2026-04-01
10403	MAIN	85.00	75.00	8500.00	2026-03-28
10404	MAIN	450.00	420.00	45000.00	2026-04-02
10405	WH01	120.00	100.00	12000.00	2026-03-25
10419	MAIN	280.00	250.00	28000.00	2026-04-01

3.2 Purchase Order Data

Why Critical: Essential for supplier performance analysis, delivery tracking, and procurement cycle optimization.

Required Fields

Field Name	Infor Column	Data Type	Example	Business Use
PO_Number	PO_CODE	VARCHAR(20)	PO-2025-001234	Order identifier
PO_Line	PO_RELEASE	INTEGER	1, 2, 3	Line number
Material_Code	ITEM	VARCHAR(50)	10401	Material ordered
Supplier_Code	VENDOR	VARCHAR(20)	SUPP-001	Supplier ID
Supplier_Name	VENDOR_NAME	VARCHAR(100)	ABC Suppliers Ltd	Supplier name
Order_Quantity	QTY_ORDERED	DECIMAL(15,3)	1000.000	Ordered qty
Received_Quantity	QTY_RECEIVED	DECIMAL(15,3)	850.000	Already received
Outstanding_Quantity	QTY_OUTSTANDING	DECIMAL(15,3)	150.000	Still pending
Unit_Price	UNIT_PRICE	DECIMAL(15,2)	100.00	Price per unit
Currency	CURRENCY	VARCHAR(3)	USD, INR	Currency code
Order_Date	PO_DATE	DATE	2026-03-01	When PO created
Promised_Delivery_Date	DUE_DATE	DATE	2026-03-15	Supplier commitment
Actual_Delivery_Date	ACTUAL_DATE	DATE	2026-03-18	When received
PO_Status	PO_STATUS	VARCHAR(10)	O, C, P	Open/Closed/Partial
GRN_Number	GRN_NUMBER	VARCHAR(20)	GRN-2025-005678	Goods receipt note
GRN_Date	GRN_DATE	DATE	2026-03-18	Receipt date
Delivery_Variance_Days	CALCULATED	INTEGER	3	Actual - Promised

Infor Tables

Primary Table: `POLINE` (Purchase Order Line)

```sql
SELECT
    p.PO_CODE as po_number,
    p.PO_RELEASE as po_line,
    p.ITEM as material_code,
    p.VENDOR as supplier_code,
    p.QTY_ORDERED as order_quantity,
    p.QTY_RECEIVED as received_quantity,
    (p.QTY_ORDERED - p.QTY_RECEIVED) as outstanding_quantity,
    p.UNIT_PRICE as unit_price,
    p.CURRENCY as currency,
    p.PO_DATE as order_date,
    p.DUE_DATE as promised_delivery_date,
    p.PO_STATUS as po_status,
    p.COMPANY as plant_code
FROM POLINE p
WHERE p.COMPANY = '21C'
  AND p.PO_DATE >= DATEADD(year, -2, GETDATE())  -- Last 2 years
ORDER BY p.PO_DATE DESC
```

Supporting Table: `APVENDOR` (Vendor Master)

```sql
SELECT
    v.VENDOR as supplier_code,
    v.NAME as supplier_name,
    v.CONTACT as contact_person,
    v.EMAIL as email,
    v.PHONE as phone,
    v.ADDRESS as address,
    v.PAYMENT_TERMS as payment_terms,
    v.CURRENCY as currency,
    v.ACTIVE_FLAG as is_active
FROM APVENDOR v
WHERE v.ACTIVE_FLAG = 'Y'
```

GRN Data: `ICDISTRIBUTION` (Receipts)

```sql
```

```sql
SELECT
    d.DOCUMENT_NUM as grn_number,
    d.TRANS_DATE as grn_date,
    d.ITEM as material_code,
    d.QUANTITY as received_qty,
    d.REF_NUM as po_number
FROM ICDISTRIBUTION d
WHERE d.COMPANY = '21C'
  AND d.TRANS_TYPE = 'R'  -- Receipts only
  AND d.TRANS_DATE >= DATEADD(month, -6, GETDATE())
```

Calculated Metrics

```sql
-- Delivery Performance
WITH po_performance AS (
    SELECT
        p.VENDOR,
        COUNT(*) as total_pos,
        SUM(CASE WHEN p.ACTUAL_DATE <= p.DUE_DATE THEN 1 ELSE 0 END) as on_time_deli
        SUM(CASE WHEN p.ACTUAL_DATE > p.DUE_DATE THEN 1 ELSE 0 END) as late_deliveri
        AVG(DATEDIFF(day, p.DUE_DATE, p.ACTUAL_DATE)) as avg_delay_days
    FROM POLINE p
    WHERE p.PO_STATUS = 'C'  -- Closed/completed POs
      AND p.ACTUAL_DATE IS NOT NULL
    GROUP BY p.VENDOR
)
SELECT * FROM po_performance
```

Data Refresh Strategy

- **Frequency:** Daily (evening batch)
- **Scope:** Last 2 years of POs (for trend analysis)
- **Incremental Load:** Only new/modified POs since last run
- **Retention:** Archive POs older than 5 years

3.3 Supplier Master Data

Why Critical: Links POs to suppliers, enables supplier scorecarding and vendor management.

Required Fields

Field Name	Infor Column	Data Type	Example	Business Use
Supplier_Code	VENDOR	VARCHAR(20)	SUPP-001	Unique supplier ID
Supplier_Name	NAME	VARCHAR(100)	ABC Suppliers Ltd	Legal name
Contact_Person	CONTACT	VARCHAR(50)	John Doe	Primary contact
Email	EMAIL	VARCHAR(100)	john@abc.com	Contact email
Phone	PHONE	VARCHAR(20)	+91-9999999999	Contact phone
Address	ADDRESS	VARCHAR(200)	123 Main St, Delhi	Full address
Payment_Terms	PAYMENT_TERMS	VARCHAR(20)	NET30, NET60	Payment terms
Currency	CURRENCY	VARCHAR(3)	INR, USD	Default currency
Is_Active	ACTIVE_FLAG	CHAR(1)	Y/N	Active status
Supplier_Type	SUPPLIER_TYPE	VARCHAR(20)	Local, Import	Supplier category
Default_Lead_Time	LEAD_TIME	INTEGER	15	Standard lead time
Rating	SUPPLIER_RATING	DECIMAL(3,2)	4.50	Performance rating (0-5)

Infor Table Query

```sql
SELECT
    v.VENDOR as supplier_code,
    v.NAME as supplier_name,
    v.CONTACT as contact_person,
    v.EMAIL as email,
    v.PHONE as phone,
    v.ADDRESS as address,
    v.PAYMENT_TERMS as payment_terms,
    v.CURRENCY as currency,
    v.ACTIVE_FLAG as is_active,
    v.SUPPLIER_TYPE as supplier_type,
    v.LEAD_TIME as default_lead_time
FROM APVENDOR v
ORDER BY v.NAME
```

3.4 Material Master Enrichment

Why Critical: Enables ABC analysis, value-based prioritization, and proper unit conversions.

Additional Fields Needed

Field Name	Infor Column	Data Type	Example	Business Use
Unit_of_Measure	UM	VARCHAR(10)	EA, KG, L	Measurement unit
Material_Group	ITEM_GROUP	VARCHAR(20)	BEARINGS, CHAINS	Category
Material_Category	ITEM_CATEGORY	VARCHAR(50)	Raw Material, Spare Part	Type
ABC_Classification	ABC_CODE	CHAR(1)	A, B, C	Value classification
XYZ_Classification	XYZ_CODE	CHAR(1)	X, Y, Z	Variability class
Standard_Price	STD_COST	DECIMAL(15,2)	100.00	Standard cost
Moving_Average_Price	UNIT_COST	DECIMAL(15,2)	105.50	Current avg cost
Criticality_Flag	CRITICAL_FLAG	CHAR(1)	H/M/L	Business criticality
Min_Order_Quantity	MIN_ORDER_QTY	DECIMAL(15,3)	100.000	MOQ
Max_Stock_Level	MAX_STOCK	DECIMAL(15,3)	5000.000	Max stock
Reorder_Level	REORDER_POINT	DECIMAL(15,3)	500.000	Reorder trigger
MRP_Type	MRP_TYPE	VARCHAR(10)	AUTO, MANUAL	Planning type
Procurement_Type	PROC_TYPE	CHAR(1)	E/F/X	External/In-house

Infor Query

```sql
```

```sql
SELECT
    i.ITEM as material_code,
    i.DESCRIPTION as description,
    i.UM as unit_of_measure,
    i.ITEM_GROUP as material_group,
    i.ITEM_CATEGORY as material_category,
    i.ABC_CODE as abc_class,
    i.STD_COST as standard_price,
    i.UNIT_COST as moving_average_price,
    i.CRITICAL_FLAG as criticality_flag,
    i.MIN_ORDER_QTY as min_order_quantity,
    i.MAX_STOCK as max_stock_level,
    i.REORDER_POINT as reorder_level,
    i.MRP_TYPE as mrp_type,
    i.PROC_TYPE as procurement_type
FROM ICITEM i
WHERE i.COMPANY = '21C'
  AND i.ACTIVE_FLAG = 'Y'
```

---

4. PRIORITY 2 DATA (SHOULD HAVE)

4.1 Inventory Movement History

Purpose: Monthly consumption trends, seasonal analysis, aging analysis.

Infor Table: `ICTRANS` (Transaction History)

```sql
SELECT
    t.ITEM as material_code,
    t.TRANS_TYPE as movement_type,
    t.QUANTITY as quantity,
    t.TRANS_DATE as movement_date,
    t.DOCUMENT_NUM as document_number,
    t.FROM_LOC as from_location,
    t.TO_LOC as to_location,
    MONTH(t.TRANS_DATE) as month,
    YEAR(t.TRANS_DATE) as year
FROM ICTRANS t
WHERE t.COMPANY = '21C'
  AND t.TRANS_DATE >= DATEADD(year, -3, GETDATE())  -- Last 3 years
ORDER BY t.TRANS_DATE DESC
```

Aggregated Monthly Consumption:

```sql
SELECT
    t.ITEM as material_code,
    CONCAT(YEAR(t.TRANS_DATE), '-', FORMAT(MONTH(t.TRANS_DATE), '00')) as year_month
    SUM(CASE WHEN t.TRANS_TYPE IN ('I', 'P') THEN t.QUANTITY ELSE -t.QUANTITY END) a
FROM ICTRANS t
WHERE t.COMPANY = '21C'
  AND t.TRANS_DATE >= DATEADD(year, -3, GETDATE())
GROUP BY t.ITEM, YEAR(t.TRANS_DATE), MONTH(t.TRANS_DATE)
ORDER BY t.ITEM, year_month
```

4.2 Production Order Data (If Applicable)

Purpose: Link consumption to production, identify critical materials.

Infor Table: `SFORDER` (Shop Floor Orders)

```sql
SELECT
    o.ORDER_NUM as production_order_number,
    o.ITEM as material_code,
    o.QTY_ORDERED as planned_quantity,
    o.QTY_COMPLETE as actual_quantity,
    o.START_DATE as order_start_date,
    o.END_DATE as order_end_date,
    o.ORDER_STATUS as status
FROM SFORDER o
WHERE o.COMPANY = '21C'
  AND o.START_DATE >= DATEADD(year, -1, GETDATE())
```

---

5. INFOR ACCESS & INTEGRATION

5.1 Access Methods (Ranked by Preference)

Option 1: ODBC Connection (Recommended for POC) ✅

Pros:

- Direct database access
- Real-time data
- Standard SQL interface
- Easy to set up

Cons:

- Requires network access to Infor DB
- Read-only access needed (for security)

Setup:

```python
import pyodbc

# Connection string
conn_str = """
Driver={Infor ODBC Driver};
Server=your_infor_server.company.com;
Database=INFOR_DATABASE;
UID=readonly_user;
PWD=secure_password;
TrustedConnection=no;
"""

# Connect
conn = pyodbc.connect(conn_str)

# Query
query = "SELECT * FROM ICITEM WHERE COMPANY = '21C' AND ACTIVE_FLAG = 'Y'"
df = pd.read_sql(query, conn)
```

Option 2: Infor ION API (REST) ✅

Pros:

- Cloud-friendly
- Rate-limited (prevents overload)
- OAuth security
- Scheduled batch exports

Cons:

- More complex setup
- Requires API credentials
- May have usage limits

Setup:

```python
```

```python
import requests
from requests_oauthlib import OAuth2Session

# OAuth credentials
client_id = 'your_client_id'
client_secret = 'your_client_secret'
token_url = 'https://your-infor-instance.com/oauth/token'

# Get token
auth = requests.auth.HTTPBasicAuth(client_id, client_secret)
data = {'grant_type': 'client_credentials'}
response = requests.post(token_url, auth=auth, data=data)
token = response.json()['access_token']

# API call
headers = {'Authorization': f'Bearer {token}'}
api_url = 'https://your-infor-instance.com/api/icitem'
response = requests.get(api_url, headers=headers)
data = response.json()
```

Option 3: IPA (Infor Process Automation) ⚠️

Pros:

- No coding needed
- Scheduled exports
- GUI-based configuration

Cons:

- Less flexible
- File-based (CSV/Excel)
- Manual or scheduled only (not real-time)

Setup:

1. Create IPA flow in Infor
2. Select tables and filters
3. Schedule export (e.g., daily at 6 AM)
4. Export to shared folder (CSV/Excel)
5. Python script picks up file and loads to SQLite

5.2 Data Extraction Checklist

Before you start extraction from Infor:

Week 1: Discovery & Access

- [ ] Identify Infor instance (CloudSuite, LN, M3, etc.)
- [ ] Get IT team approval for read-only database access
- [ ] Obtain credentials (ODBC or API)
- [ ] Test connection from your dev machine
- [ ] Document network/firewall requirements

Week 2: Table Mapping

- [ ] Verify table names (they may vary by Infor version)
- [ ] Document field mappings (Infor → Your schema)
- [ ] Check data types and formats
- [ ] Identify primary and foreign keys
- [ ] Test sample queries on each table

Week 3: Data Profiling

- [ ] Check row counts (ensure data exists)
- [ ] Identify null values and data quality issues
- [ ] Validate date ranges (are historical data available?)
- [ ] Check for duplicate records
- [ ] Verify company code filtering ('21C')

Week 4: ETL Development

- [ ] Build extraction scripts (one per table)
- [ ] Implement incremental load logic
- [ ] Add data validation and error handling
- [ ] Test with small data subset
- [ ] Schedule automated runs

---

6. PHASED ROLLOUT STRATEGY

Phase 1: POC (Weeks 1-4) - CURRENT STATUS ✅

Data: Static Excel files
Scope: 426 materials, 3-year consumption
Capabilities: Trend analysis, ABC/XYZ classification, basic forecasting
Deliverable: Working POC with Streamlit UI

Phase 2: Infor Integration (Weeks 5-8)

Data: Priority 1 tables (Inventory + PO + Supplier)
Scope: Real-time inventory, supplier performance
Capabilities: All questions in Section 2.1 answerable
Deliverable: Production-ready agent with live data

Phase 3: Advanced Analytics (Weeks 9-12)

Data: Priority 2 tables (Movement history + Production)
Scope: Monthly trends, forecasting, alerts
Capabilities: Proactive alerts, predictive analytics
Deliverable: Enterprise-grade analytics platform

Phase 4: Autonomous Actions (Weeks 13-16)

Data: All tables + Write access (controlled)
Scope: Auto-generate POs, reorder recommendations
Capabilities: Fully autonomous procurement agent
Deliverable: Self-managing inventory system

---

7. INFOR-SPECIFIC CONSIDERATIONS

7.1 Common Gotchas

1. **Company Code Filtering**: Always filter by `COMPANY = '21C'`
2. **Date Formats**: Infor often uses Julian dates (CYYMMDD format)
3. **Null vs Spaces**: Infor may store NULL as empty strings
4. **Unit Conversions**: Handle different UoMs (EA, KG, L, M, etc.)
5. **Active Flags**: Check `ACTIVE_FLAG = 'Y'` to exclude deleted records
6. **Case Sensitivity**: Table and column names may be case-sensitive

7.2 Performance Optimization

1. **Use Indexes**: Ensure indexes on ITEM, COMPANY, TRANS_DATE
2. **Limit Date Ranges**: Don't pull all historical data (use last 2-3 years)
3. **Batch Processing**: For large tables, use LIMIT/OFFSET or cursor pagination
4. **Caching**: Cache reference data (material master, suppliers) locally
5. **Incremental Loads**: Use `WHERE LAST_UPDATE_DATE > :last_run_date`

---

8. SAMPLE INTEGRATION CODE

8.1 Complete ETL Script

```python
```

```python
"""
Infor Data Extraction - Production Version
"""

import pyodbc
import pandas as pd
import sqlite3
from datetime import datetime, timedelta
import logging

# Setup logging
logging.basicConfig(level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

class InforDataExtractor:
    """Extract data from Infor and load to SQLite"""

    def __init__(self, infor_conn_str, sqlite_db_path):
        self.infor_conn_str = infor_conn_str
        self.sqlite_db_path = sqlite_db_path

    def extract_material_master(self):
        """Extract material master data"""
        logger.info("Extracting material master...")

        query = """
        SELECT
            i.ITEM as material_code,
            i.DESCRIPTION as description,
            i.UM as unit_of_measure,
            i.ITEM_GROUP as material_group,
            i.STD_COST as standard_price,
            i.UNIT_COST as moving_average_price,
            i.ABC_CODE as abc_class,
            i.REORDER_POINT as reorder_level,
            i.SAFETY_STOCK as safety_stock,
            i.LEAD_TIME as lead_time_days
        FROM ICITEM i
        WHERE i.COMPANY = '21C'
          AND i.ACTIVE_FLAG = 'Y'
        """

        with pyodbc.connect(self.infor_conn_str) as conn:
            df = pd.read_sql(query, conn)
```

```
        logger.info(f"Extracted {len(df)} materials")
        return df

    def extract_current_inventory(self):
        """Extract current inventory levels"""
        logger.info("Extracting current inventory...")

        query = """
        SELECT
            l.ITEM as material_code,
            l.COMPANY as plant_code,
            l.LOCATION as storage_location,
            l.ON_HAND as unrestricted_stock,
            l.COMMITTED as reserved_stock,
            (l.ON_HAND - l.COMMITTED) as available_stock,
            l.LAST_MOVE_DATE as last_movement_date,
            GETDATE() as snapshot_date
        FROM ICLOCATION l
        WHERE l.COMPANY = '21C'
          AND l.ON_HAND > 0
        """

        with pyodbc.connect(self.infor_conn_str) as conn:
            df = pd.read_sql(query, conn)

        logger.info(f"Extracted inventory for {len(df)} material-location combinatio
        return df

    def extract_purchase_orders(self, days_back=730):
        """Extract purchase orders (last 2 years)"""
        logger.info(f"Extracting purchase orders (last {days_back} days)...")

        query = f"""
        SELECT
            p.PO_CODE as po_number,
            p.PO_RELEASE as po_line,
            p.ITEM as material_code,
            p.VENDOR as supplier_code,
            v.NAME as supplier_name,
            p.QTY_ORDERED as order_quantity,
            p.QTY_RECEIVED as received_quantity,
            (p.QTY_ORDERED - p.QTY_RECEIVED) as outstanding_quantity,
            p.UNIT_PRICE as unit_price,
            p.CURRENCY as currency,
            p.PO_DATE as order_date,
            p.DUE_DATE as promised_delivery_date,
            p.PO_STATUS as po_status
```

```
        FROM POLINE p
        LEFT JOIN APVENDOR v ON p.VENDOR = v.VENDOR
        WHERE p.COMPANY = '21C'
          AND p.PO_DATE >= DATEADD(day, -{days_back}, GETDATE())
        ORDER BY p.PO_DATE DESC
        """

        with pyodbc.connect(self.infor_conn_str) as conn:
            df = pd.read_sql(query, conn)

        logger.info(f"Extracted {len(df)} purchase order lines")
        return df

    def extract_suppliers(self):
        """Extract supplier master"""
        logger.info("Extracting supplier master...")

        query = """
        SELECT
            v.VENDOR as supplier_code,
            v.NAME as supplier_name,
            v.CONTACT as contact_person,
            v.EMAIL as email,
            v.PHONE as phone,
            v.PAYMENT_TERMS as payment_terms,
            v.CURRENCY as currency,
            v.ACTIVE_FLAG as is_active
        FROM APVENDOR v
        WHERE v.ACTIVE_FLAG = 'Y'
        """

        with pyodbc.connect(self.infor_conn_str) as conn:
            df = pd.read_sql(query, conn)

        logger.info(f"Extracted {len(df)} suppliers")
        return df

    def load_to_sqlite(self, df, table_name, if_exists='replace'):
        """Load DataFrame to SQLite"""
        logger.info(f"Loading to SQLite table: {table_name}")

        with sqlite3.connect(self.sqlite_db_path) as conn:
            df.to_sql(table_name, conn, if_exists=if_exists, index=False)

        logger.info(f"Loaded {len(df)} rows to {table_name}")

    def run_full_extraction(self):
```

```python
        """Run complete ETL pipeline"""
        logger.info("="*60)
        logger.info("Starting full data extraction from Infor")
        logger.info("="*60)

        try:
            # Extract
            df_materials = self.extract_material_master()
            df_inventory = self.extract_current_inventory()
            df_pos = self.extract_purchase_orders()
            df_suppliers = self.extract_suppliers()

            # Load
            self.load_to_sqlite(df_materials, 'material_master')
            self.load_to_sqlite(df_inventory, 'inventory_current')
            self.load_to_sqlite(df_pos, 'purchase_orders')
            self.load_to_sqlite(df_suppliers, 'supplier_master')

            logger.info("="*60)
            logger.info("✅ Data extraction completed successfully!")
            logger.info("="*60)

        except Exception as e:
            logger.error(f"❌ Error during extraction: {str(e)}")
            raise

# Usage
if __name__ == "__main__":
    # Configuration
    INFOR_CONN_STR = """
    Driver={Infor ODBC Driver};
    Server=your_infor_server;
    Database=INFOR_DB;
    UID=readonly_user;
    PWD=password;
    """

    SQLITE_DB = "procurement_production.db"

    # Run extraction
    extractor = InforDataExtractor(INFOR_CONN_STR, SQLITE_DB)
    extractor.run_full_extraction()
```

---

9. SUCCESS METRICS

Data Quality Metrics

- [ ] **Completeness**: >95% of expected records loaded
- [ ] **Accuracy**: Zero critical errors in sample validation
- [ ] **Freshness**: Data <24 hours old
- [ ] **Consistency**: No duplicate records

Business Metrics

- [ ] **Query Coverage**: 90%+ of business questions answerable
- [ ] **Response Time**: <5 seconds for 95% of queries
- [ ] **User Adoption**: 80%+ of procurement team using agent
- [ ] **Time Savings**: 50% reduction in manual reporting time

---

10. NEXT ACTIONS

Immediate (This Week)

1. **Schedule meeting with IT team**
    - Topic: Infor database access request
    - Attendees: You, IT DBA, Security team
    - Agenda: Read-only access for tables in Section 3
2. **Document requirements**
    - List all Infor tables needed
    - Specify fields required
    - Define refresh frequency
    - Security/compliance constraints
3. **Set up development environment**
    - Install ODBC drivers
    - Test connection to Infor
    - Create test database

Week 1-2

1. Extract Priority 1 data (Section 3.1-3.4)
2. Validate data quality
3. Load to production database
4. Test agent with real Infor data

Week 3-4

1. Implement incremental load
2. Set up automated scheduling
3. Add monitoring and alerts
4. Prepare for production deployment

Document Owner: Ayush (Senior Consultant, EXL Analytics)
Last Updated: April 2026
Next Review: After Infor access granted